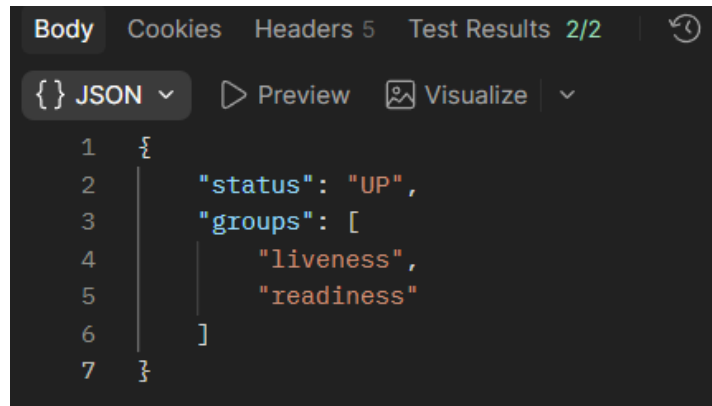


1. Levantando con compose UP y verificando con compose ps.

```
#20 resolving provenance for metadata file
#20 DONE 0.0s
[+] up 5/5
✓ Image mexi-banco-app Built 8.2s
✓ Network mexibanco Created 0.0s
✓ Volume mexi-banco_mexibanco_datos Created 0.0s
✓ Container mexi-banco-db-1 Healthy 5.8s
✓ Container mexi-banco-app-1 Started 5.8s
PS C:\Users\PC\Desktop\MexiBanco\mexi-banco> docker compose ps
NAME                IMAGE                COMMAND                  SERVICE    CREATED      STATUS              PORTS
mexi-banco-app-1    mexi-banco-app       "java -jar app.jar"     app        15 seconds ago Up 9 seconds (health: starting) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine   "docker-entrypoint.s..." db          15 seconds ago Up 15 seconds (healthy) 5432/tcp
PS C:\Users\PC\Desktop\MexiBanco\mexi-banco>
```

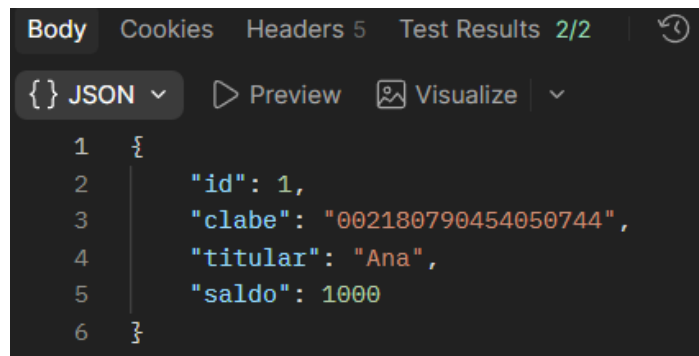
2.1 Salud de la app: confirmé que la app está levantada y responde UP.



The screenshot shows a REST client interface with the 'Body' tab selected. The response is in JSON format, showing a status of 'UP' and a list of health groups: 'liveness' and 'readiness'.

```
{
  "status": "UP",
  "groups": [
    "liveness",
    "readiness"
  ]
}
```

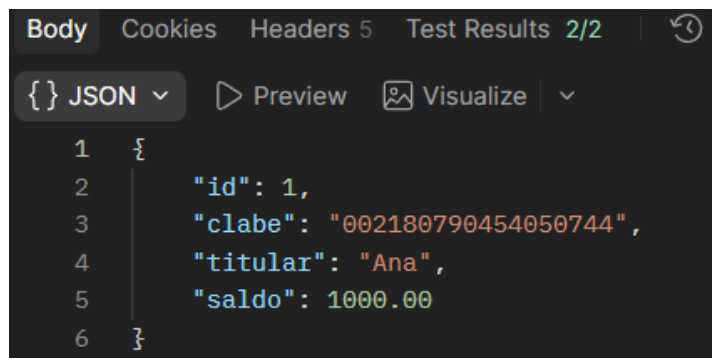
2.2 Abrir cuenta origen (1000): creé la cuenta origen con saldo inicial de \$1000.



The screenshot shows a REST client interface with the 'Body' tab selected. The response is in JSON format, showing the details of a newly created account with ID 1, clabe 002180790454050744, titular Ana, and saldo 1000.

```
{
  "id": 1,
  "clabe": "002180790454050744",
  "titular": "Ana",
  "saldo": 1000
}
```

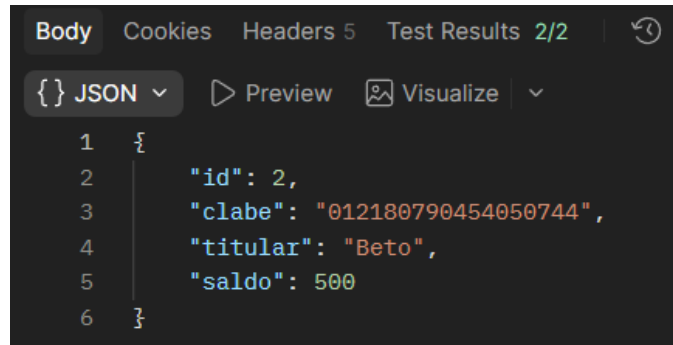
2.3 Consultar cuenta origen: verifiqué que la cuenta quedó guardada con sus \$1000.



The screenshot shows a REST client interface with the 'Body' tab selected. The response is in JSON format, showing the details of a queried account with ID 1, clabe 002180790454050744, titular Ana, and saldo 1000.00.

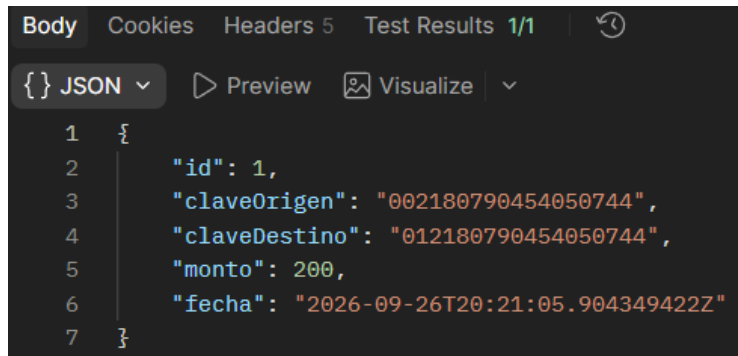
```
{
  "id": 1,
  "clabe": "002180790454050744",
  "titular": "Ana",
  "saldo": 1000.00
}
```

2.4 Abrir cuenta destino (500): creé una segunda cuenta con \$500 para recibir la transferencia.



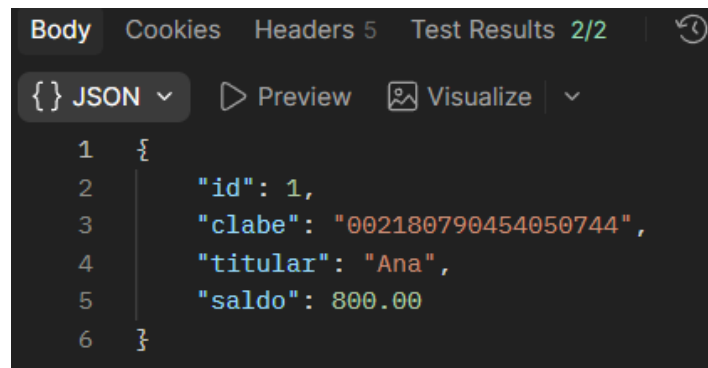
```
Body Cookies Headers 5 Test Results 2/2
{ } JSON Preview Visualize
1 {
2   "id": 2,
3   "clabe": "012180790454050744",
4   "titular": "Beto",
5   "saldo": 500
6 }
```

2.5 Transferir 200 de origen a destino: hice una transferencia interna de \$200, que el servidor registró con su propio id.



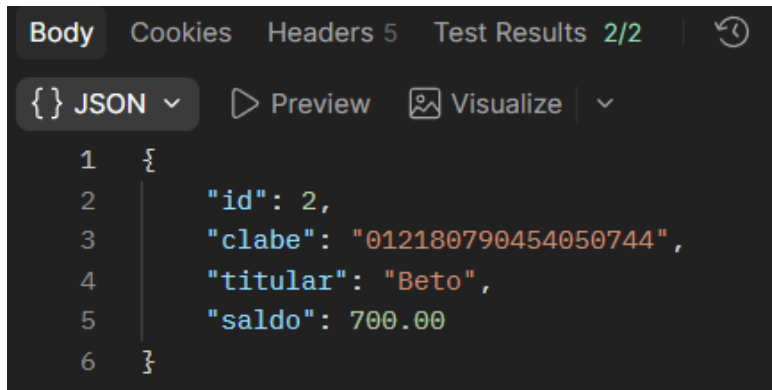
```
Body Cookies Headers 5 Test Results 1/1
{ } JSON Preview Visualize
1 {
2   "id": 1,
3   "claveOrigen": "002180790454050744",
4   "claveDestino": "012180790454050744",
5   "monto": 200,
6   "fecha": "2026-09-26T20:21:05.904349422Z"
7 }
```

2.6 Consultar origen (800): comprobé que al origen se le cobraron los \$200 y quedó en \$800.



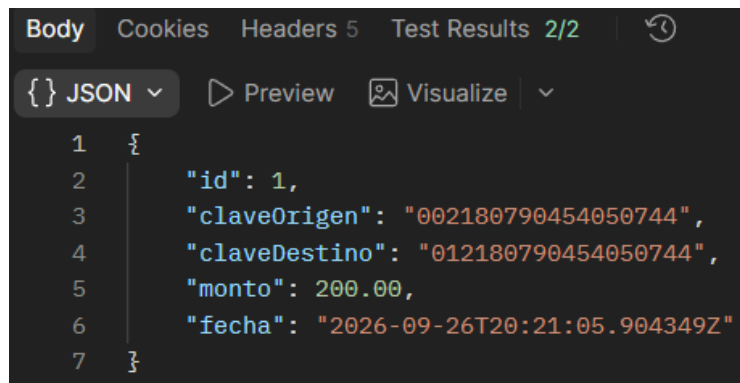
```
Body Cookies Headers 5 Test Results 2/2
{ } JSON Preview Visualize
1 {
2   "id": 1,
3   "clabe": "002180790454050744",
4   "titular": "Ana",
5   "saldo": 800.00
6 }
```

2.7 **Consultar destino (700):** comprobé que al destino le llegaron los \$200 y quedó en \$700.



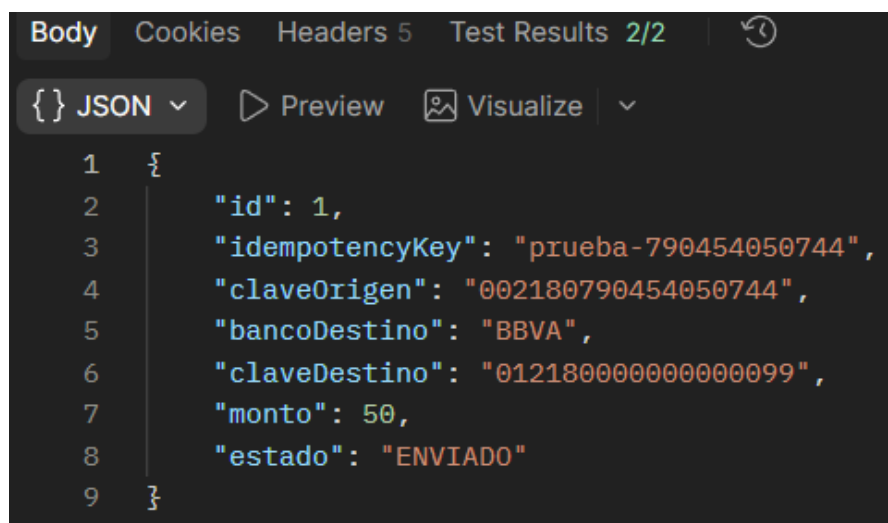
```
Body Cookies Headers 5 Test Results 2/2
{ } JSON Preview Visualize
1 {
2   "id": 2,
3   "clabe": "012180790454050744",
4   "titular": "Beto",
5   "saldo": 700.00
6 }
```

2.8 **Consultar la transferencia:** consulté la transferencia por su id y vi origen, destino, monto y fecha.



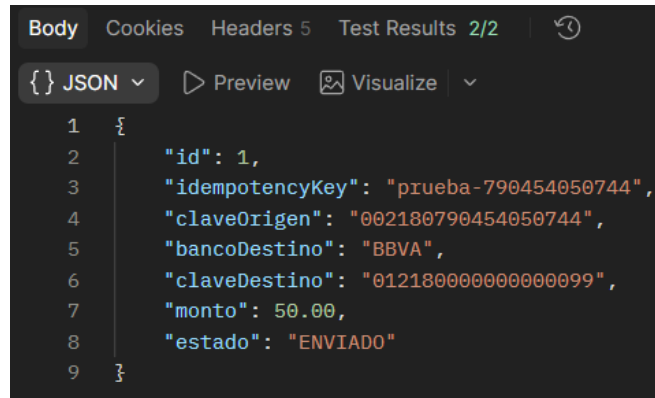
```
Body Cookies Headers 5 Test Results 2/2
{ } JSON Preview Visualize
1 {
2   "id": 1,
3   "claveOrigen": "002180790454050744",
4   "claveDestino": "012180790454050744",
5   "monto": 200.00,
6   "fecha": "2026-09-26T20:21:05.904349Z"
7 }
```

2.9 **SPEI de 50 (primer envío):** mandé un SPEI de \$50 a otro banco con una Idempotency-Key, y se cobró.



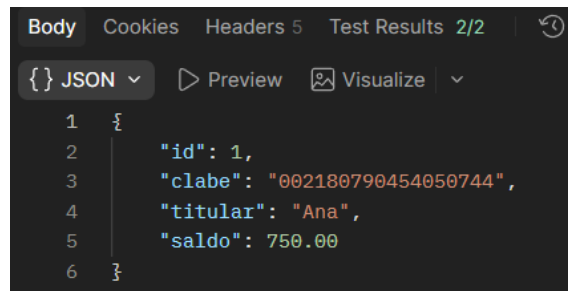
```
Body Cookies Headers 5 Test Results 2/2
{ } JSON Preview Visualize
1 {
2   "id": 1,
3   "idempotencyKey": "prueba-790454050744",
4   "claveOrigen": "002180790454050744",
5   "bancoDestino": "BBVA",
6   "claveDestino": "012180000000000099",
7   "monto": 50,
8   "estado": "ENVIADO"
9 }
```

2.10 SPEI de 50 (reintento, misma clave): repetí el mismo SPEI con la misma clave, y el sistema regresó la operación original sin volver a cobrar.



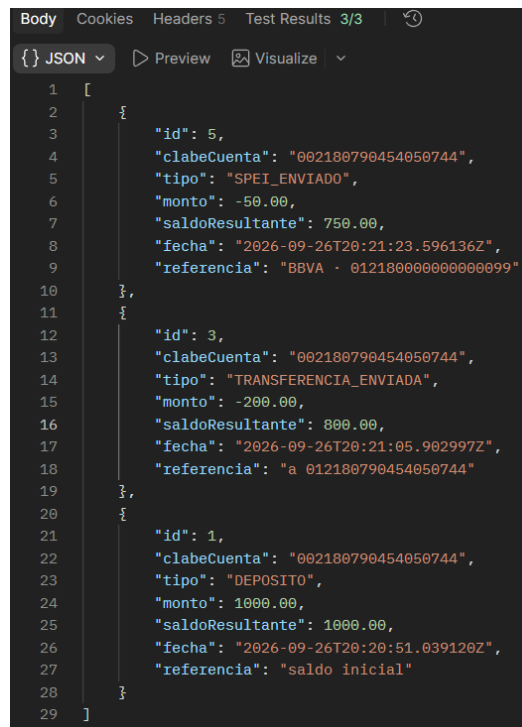
```
Body Cookies Headers 5 Test Results 2/2
{ } JSON Preview Visualize
1 {
2   "id": 1,
3   "idempotencyKey": "prueba-790454050744",
4   "claveOrigen": "002180790454050744",
5   "bancoDestino": "BBVA",
6   "claveDestino": "012180000000000099",
7   "monto": 50.00,
8   "estado": "ENVIADO"
9 }
```

2.11 Consultar origen (750, no 700): confirmé que el SPEI se cobró solo una vez: $\$800 - \$50 = \$750$.



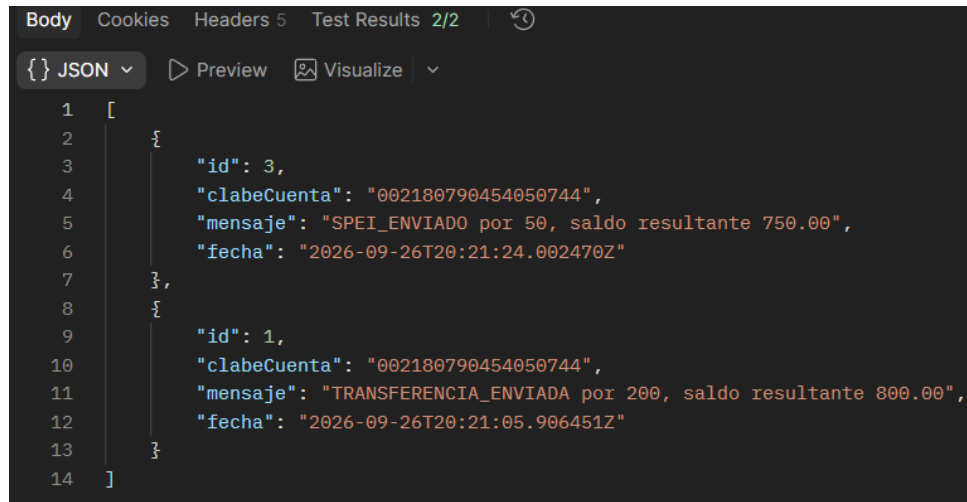
```
Body Cookies Headers 5 Test Results 2/2
{ } JSON Preview Visualize
1 {
2   "id": 1,
3   "clabe": "002180790454050744",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```

2.12 Estado de cuenta del origen: revisé los movimientos del origen, del más reciente al más viejo (el cargo de la transferencia y el del SPEI).



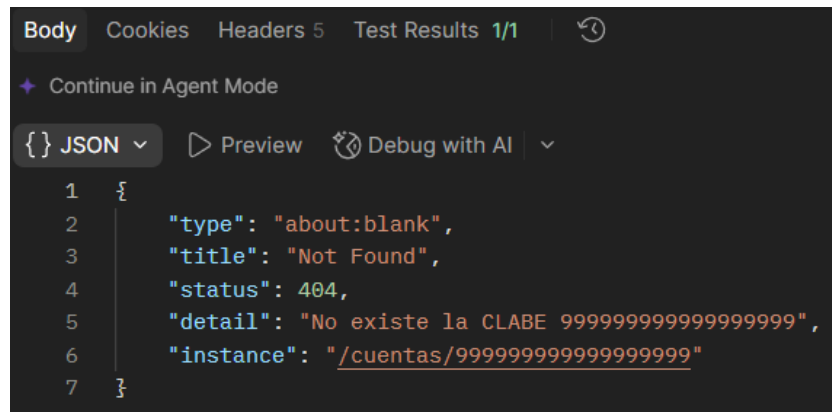
```
Body Cookies Headers 5 Test Results 3/3
{ } JSON Preview Visualize
1 [
2   {
3     "id": 5,
4     "clabeCuenta": "002180790454050744",
5     "tipo": "SPEI_ENVIADO",
6     "monto": -50.00,
7     "saldoResultante": 750.00,
8     "fecha": "2026-09-26T20:21:23.596136Z",
9     "referencia": "BBVA - 012180000000000099"
10  },
11  {
12    "id": 3,
13    "clabeCuenta": "002180790454050744",
14    "tipo": "TRANSFERENCIA_ENVIADA",
15    "monto": -200.00,
16    "saldoResultante": 800.00,
17    "fecha": "2026-09-26T20:21:05.902997Z",
18    "referencia": "a 012180790454050744"
19  },
20  {
21    "id": 1,
22    "clabeCuenta": "002180790454050744",
23    "tipo": "DEPOSITO",
24    "monto": 1000.00,
25    "saldoResultante": 1000.00,
26    "fecha": "2026-09-26T20:20:51.039120Z",
27    "referencia": "saldo inicial"
28  }
29 ]
```

2.13 **Avisos del origen:** revisé las notificaciones que se generaron automáticamente por cada movimiento.



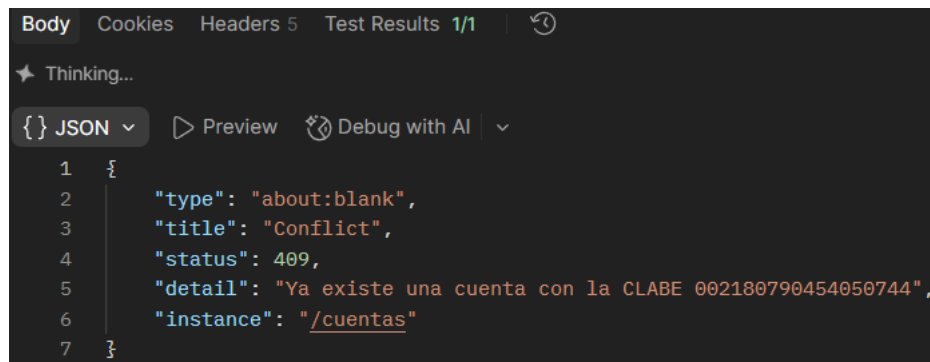
```
Body Cookies Headers 5 Test Results 2/2
{ } JSON Preview Visualize
1 [
2   {
3     "id": 3,
4     "clabeCuenta": "002180790454050744",
5     "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
6     "fecha": "2026-09-26T20:21:24.002470Z"
7   },
8   {
9     "id": 1,
10    "clabeCuenta": "002180790454050744",
11    "mensaje": "TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00",
12    "fecha": "2026-09-26T20:21:05.906451Z"
13  }
14 ]
```

2.14 **CLABE que no existe (404):** consulté una cuenta inexistente y respondió 404 No encontrado.



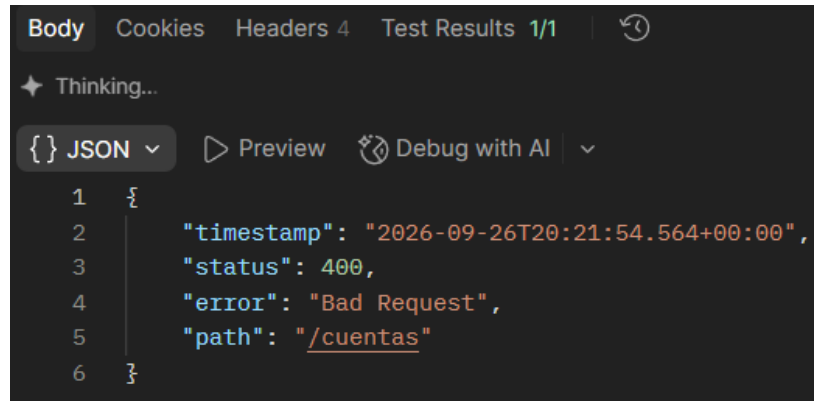
```
Body Cookies Headers 5 Test Results 1/1
Continue in Agent Mode
{ } JSON Preview Debug with AI
1 {
2   "type": "about:blank",
3   "title": "Not Found",
4   "status": 404,
5   "detail": "No existe la CLABE 999999999999999999",
6   "instance": "/cuentas/999999999999999999"
7 }
```

2.15 **CLABE repetida (409):** intenté abrir una cuenta con una CLABE ya registrada y respondió 409 Conflicto.



```
Body Cookies Headers 5 Test Results 1/1
Thinking...
{ } JSON Preview Debug with AI
1 {
2   "type": "about:blank",
3   "title": "Conflict",
4   "status": 409,
5   "detail": "Ya existe una cuenta con la CLABE 002180790454050744",
6   "instance": "/cuentas"
7 }
```

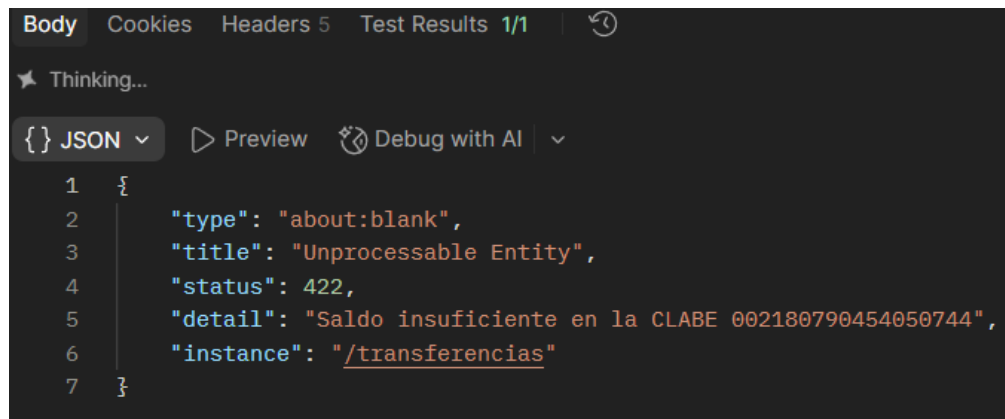
2.16 **Cuerpo sin clabe (400):** mandé una petición sin un campo obligatorio y respondió 400 Petición inválida.



The screenshot shows a REST client interface with the 'Body' tab selected. The response is in JSON format, indicating a 400 Bad Request error. The JSON body contains the following fields:

```
1 {
2   "timestamp": "2026-09-26T20:21:54.564+00:00",
3   "status": 400,
4   "error": "Bad Request",
5   "path": "/cuentas"
6 }
```

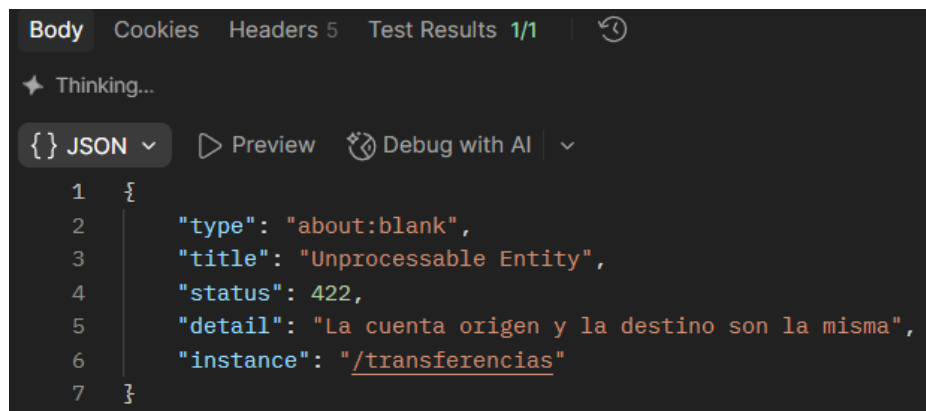
2.17 **Saldo insuficiente (422):** intenté transferir más de lo que tenía la cuenta y el sistema lo rechazó con 422.



The screenshot shows a REST client interface with the 'Body' tab selected. The response is in JSON format, indicating a 422 Unprocessable Entity error. The JSON body contains the following fields:

```
1 {
2   "type": "about:blank",
3   "title": "Unprocessable Entity",
4   "status": 422,
5   "detail": "Saldo insuficiente en la CLABE 002180790454050744",
6   "instance": "/transferencias"
7 }
```

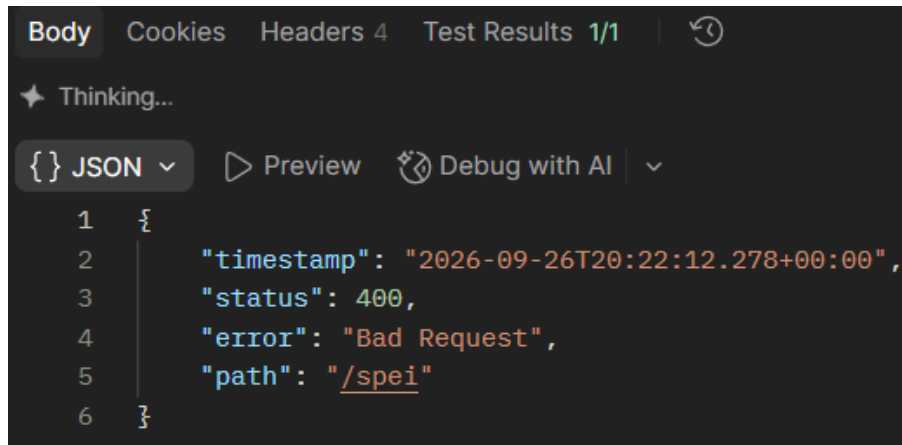
2.18 **Transferir a la misma cuenta (422):** intenté transferir de una cuenta a sí misma y la regla de negocio lo rechazó con 422.



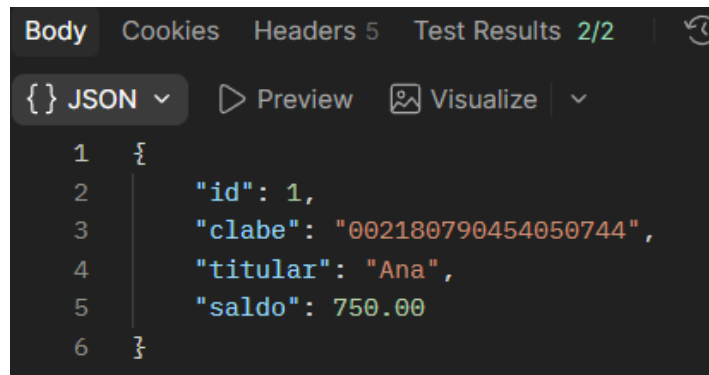
The screenshot shows a REST client interface with the 'Body' tab selected. The response is in JSON format, indicating a 422 Unprocessable Entity error. The JSON body contains the following fields:

```
1 {
2   "type": "about:blank",
3   "title": "Unprocessable Entity",
4   "status": 422,
5   "detail": "La cuenta origen y la destino son la misma",
6   "instance": "/transferencias"
7 }
```

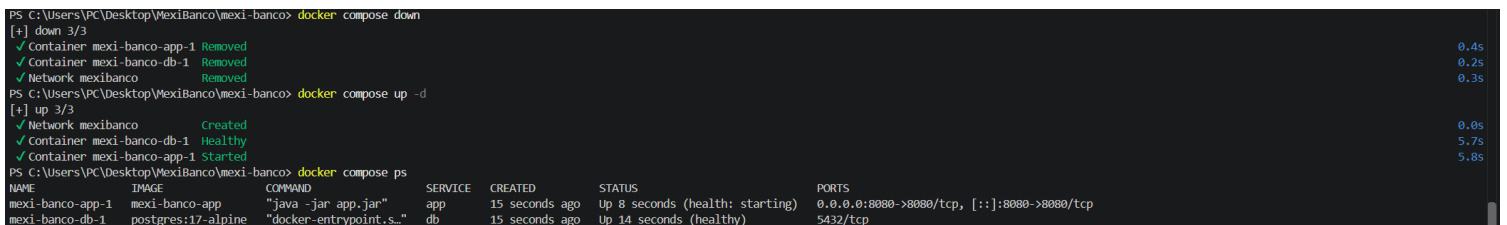
2.19 **SPEI sin Idempotency-Key (400)**: mandé un SPEI sin la cabecera obligatoria y respondió 400.

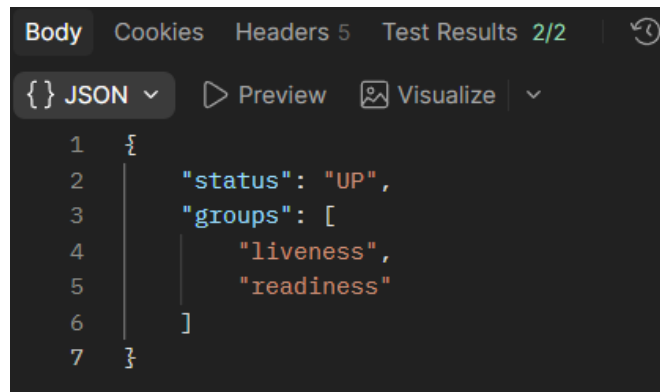


2.20 **Los errores no movieron el saldo (750)**: verifiqué que ninguna petición fallida cambió el saldo, que sigue en \$750.

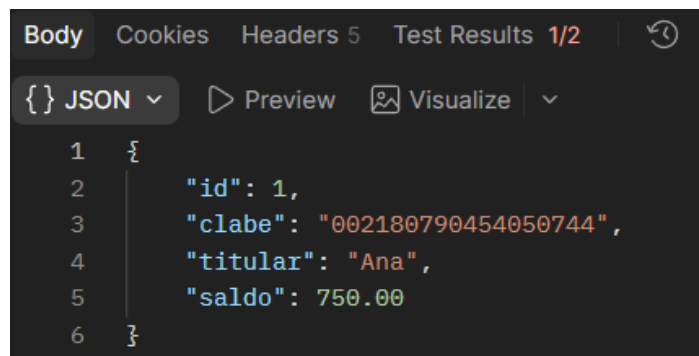


3. **Apagando, volviendo a levantar y consultar la cuenta en Postman sin borrar el volumen.**





```
Body Cookies Headers 5 Test Results 2/2
{ } JSON Preview Visualize
1 {
2   "status": "UP",
3   "groups": [
4     "liveness",
5     "readiness"
6   ]
7 }
```

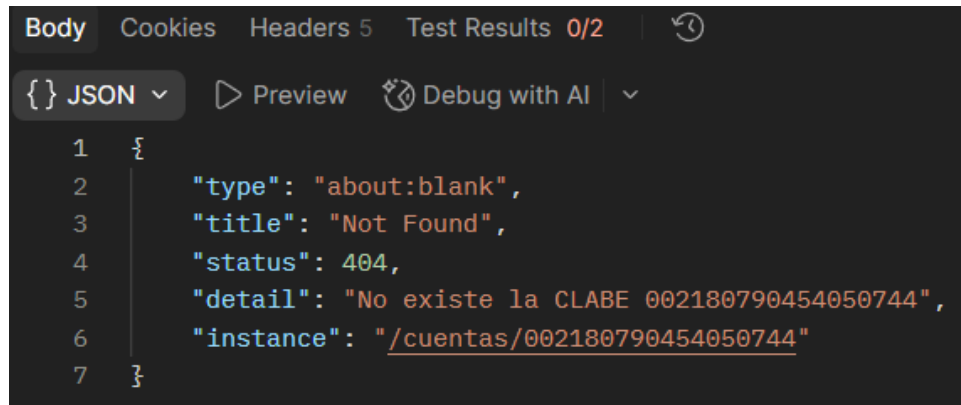


```
Body Cookies Headers 5 Test Results 1/2
{ } JSON Preview Visualize
1 {
2   "id": 1,
3   "clabe": "002180790454050744",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```

Es porque down borra los contenedores y la red, pero no los volúmenes. Los datos de Postgres viven en el volumen `mexibanco_datos`, que está montado en `/var/lib/postgresql/data`, así que siguen ahí.

3.1 Apagando, volviendo a levantar y consultar la cuenta en Postman borrando el volumen.

```
PS C:\Users\PC\Desktop\MexiBanco\mexi-banco> docker compose down -v
[+] down 4/4
✓ Container mexi-banco-app-1      Removed
✓ Container mexi-banco-db-1      Removed
✓ Volume mexi-banco_mexibanco_datos Removed
✓ Network mexibanco              Removed
PS C:\Users\PC\Desktop\MexiBanco\mexi-banco> docker compose up -d
[+] up 4/4
✓ Volume mexi-banco_mexibanco_datos Created
✓ Network mexibanco              Created
✓ Container mexi-banco-db-1      Healthy
✓ Container mexi-banco-app-1     Started
PS C:\Users\PC\Desktop\MexiBanco\mexi-banco>
```

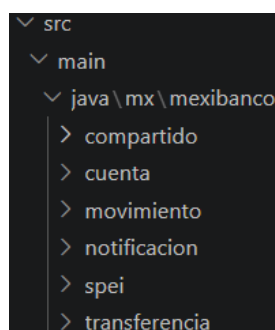


```
Body Cookies Headers 5 Test Results 0/2
[{} JSON] Preview Debug with AI
1 {
2   "type": "about:blank",
3   "title": "Not Found",
4   "status": 404,
5   "detail": "No existe la CLABE 002180790454050744",
6   "instance": "/cuentas/002180790454050744"
7 }
```

Es porque -v borra el volumen nombrado y Postgres arranca con una base vacía. La app vuelve a crear las tablas vacías porque tiene ddl-auto: update.

4. Revisando los módulos:

- **compartido:** tiene las excepciones de negocio y ManejadorDeErrores, que es el único lugar donde esas excepciones se convierten en respuestas HTTP (404, 409, 422).
- **cuenta:** abre y consulta cuentas, y es el único módulo que modifica saldos (cargar y abonar). Los demás módulos le piden a él que mueva el dinero.
- **movimiento:** es el libro contable (ledger). Solo registra y consulta asientos; no permite editarlos ni borrarlos.
- **notificacion:** genera y guarda un aviso por cada movimiento y lo escribe en el log. Más adelante se conectará a un broker como RabbitMQ o Kafka.
- **spei:** hace las transferencias a otro banco usando Idempotency-Key. Primero reserva la solicitud (estado PROCESANDO) y luego cobra (estado ENVIADO), así que un reintento no cobra dos veces.
- **transferencia:** hace las transferencias internas entre cuentas del mismo banco. Cargo, abono, movimientos y avisos van en una sola transacción: o se guardan todos o ninguno.



```
src
├── main
│   └── java\mx\mexibanco
│       ├── compartido
│       ├── cuenta
│       ├── movimiento
│       ├── notificacion
│       ├── spei
│       └── transferencia
```

5. Compose y Dockerfile

En **docker-compose.yml**:

- **db**: usa postgres:17-alpine. Su healthcheck es pg_isready.
- **app**: se construye con build: . y publica el puerto 8080:8080. Encuentra la base por nombre con DB_HOST: db, sin IP. Tiene depends_on con service_healthy, así que espera a que la base esté lista de verdad, y su healthcheck consulta /actuator/health.
- **Red mexibanco y volumen mexibanco_datos**: este volumen es el mismo del Paso 4.

En el **Dockerfile**:

- **Etapas 1 (builder, con JDK)**: copia primero pom.xml y mvnw y descarga las dependencias. Así esa capa queda en caché mientras no cambie el pom. Después copia src y compila el .jar.
- **Etapas 2 (solo JRE)**: copia únicamente el .jar y lo ejecuta. La imagen final queda más ligera y sin las herramientas de compilación.